Hotel Booking System - Database Design

Proposed schema | PostgreSQL • FastAPI • Next.js

1. Scope and assumptions

- Multiple independently registered hotels use one shared database. Every hotel-owned record is linked to a hotel_id so the API can enforce tenant isolation.
- A user account can register hotels and/or be assigned to manage multiple hotels. hotel_members stores these permissions.
- A booking can include multiple requested rooms. Each booking_room row represents one requested room; room_id can remain NULL until a physical room is assigned.
- Prices are stored as NUMERIC and the booked nightly rate is snapshotted in booking_rooms so later rate changes do not rewrite booking history.
- Availability must be checked for overlapping date ranges in a transaction. Use PostgreSQL exclusion constraints or locking to prevent concurrent double-booking of the same physical room.
- Authentication credentials are represented by password_hash; store only a strong password hash, never plaintext passwords. Payment provider secrets must remain in environment variables.

2.1. Table: users

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	User identifier
full_name	VARCHAR(120)	NOT NULL	Display name
email	CITEXT	UNIQUE, NOT NULL	Login email; enable citext extension or use a unique lower(email) index
password_hash	TEXT	NOT NULL	Hashed password
phone	VARCHAR(25)	NULL	Contact number
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Account creation time

2.2. Table: hotels

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Hotel identifier
name	VARCHAR(180)	NOT NULL	Public hotel name
description	TEXT	NULL	Hotel description
address_line	VARCHAR(255)	NOT NULL	Street address
city	VARCHAR(100)	NOT NULL	City
state	VARCHAR(100)	NULL	State/region
country	VARCHAR(100)	NOT NULL	Country
postal_code	VARCHAR(20)	NULL	Postal/ZIP code
timezone	VARCHAR(64)	NOT NULL	IANA timezone, e.g. Asia/Kolkata
status	VARCHAR(20)	NOT NULL DEFAULT 'pending'	pending, active, suspended
registered_by	BIGINT	FK → users.id, NOT NULL	User who registered the hotel
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Registration time

2.3. Table: hotel_members

Column	Data type	Key / constraint	Purpose
hotel_id	BIGINT	PK, FK → hotels.id	Hotel being managed
user_id	BIGINT	PK, FK → users.id	Member account
role	VARCHAR(20)	NOT NULL	owner, manager, staff
joined_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Membership start

2.4. Table: room_types

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Room-type identifier
hotel_id	BIGINT	FK → hotels.id, NOT NULL	Owning hotel
name	VARCHAR(100)	NOT NULL	e.g. Deluxe Double
description	TEXT	NULL	Room-type details
max_guests	SMALLINT	NOT NULL CHECK > 0	Maximum occupancy
base_price	NUMERIC(12,2)	NOT NULL CHECK >= 0	Default nightly price

created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Creation time
Constraint	—	UNIQUE (hotel_id, name)	Room-type names unique within a hotel

2.5. Table: rooms

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Physical room identifier
hotel_id	BIGINT	FK → hotels.id, NOT NULL	Owning hotel; must match room type's hotel
room_type_id	BIGINT	FK → room_types.id, NOT NULL	Room category
room_number	VARCHAR(20)	NOT NULL	Room number/code
floor	SMALLINT	NULL	Floor number
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active, maintenance, inactive
Constraint	—	UNIQUE (hotel_id, room_number)	Room number unique within a hotel

2.6. Table: bookings

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Booking identifier
user_id	BIGINT	FK → users.id, NOT NULL	Customer who made booking
check_in	DATE	NOT NULL	Arrival date
check_out	DATE	NOT NULL CHECK check_out > check_in	Departure date; checkout is exclusive
guest_count	SMALLINT9	NOT NULL CHECK > 0	Total guests
status	VARCHAR(20)	NOT NULL DEFAULT 'pending'	pending, confirmed, cancelled, completed
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Booking creation time

2.7. Table: booking_rooms

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Requested room line identifier
booking_id	BIGINT	FK → bookings.id, NOT NULL	Parent booking
room_type_id	BIGINT	FK → room_types.id, NOT NULL	Requested room category
room_id	BIGINT	FK → rooms.id, NULL	Assigned physical room; nullable before allocation
nightly_rate	NUMERIC(12,2)	NOT NULL CHECK >= 0	Rate agreed at booking time
guest_count	SMALLINT	NOT NULL CHECK > 0	Guests for this room
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Line creation time

2.8. Table: payments

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Payment identifier
booking_id	BIGINT	FK → bookings.id, NOT NULL	Booking being paid for
provider	VARCHAR(40)	NOT NULL	Payment gateway name
provider_reference	VARCHAR(160)	UNIQUE, NULL	Gateway transaction reference
amount	NUMERIC(12,2)	NOT NULL CHECK > 0	Amount attempted/paid
currency	CHAR(3)	NOT NULL DEFAULT 'INR'	ISO currency code
status	VARCHAR(20)	NOT NULL	initiated, successful, failed, refunded
paid_at	TIMESTAMPTZ	NULL	Successful payment timestamp
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Payment record creation

2.9. Table: reviews

Column	Data type	Key / constraint	Purpose
id	BIGINT	PK	Review identifier
booking_id	BIGINT	UNIQUE, FK → bookings.id	One review per booking
user_id	BIGINT	FK → users.id, NOT NULL	Review author; must match booking customer
hotel_id	BIGINT	FK → hotels.id, NOT NULL	Reviewed hotel
rating	SMALLINT	NOT NULL CHECK 1..5	Rating
comment	TEXT	NULL	Written feedback
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Review creation time

3. Relationships

- users 1:N hotels through hotels.registered_by (registrant).
- users M:N hotels through hotel_members; role is an attribute of the membership relationship.
- hotels 1:N room_types and hotels 1—N rooms.
- room_types 1:N rooms; a room belongs to one room type.
- users 1:N bookings; each booking belongs to one customer.
- bookings 1:N booking_rooms; each booking-room line requests one room of a selected type and may later be assigned a physical room.
- room_types 1:N booking_rooms; rooms 1—N booking_rooms over time (room_id is nullable until assignment).
- bookings 1:N payments; a booking may have multiple payment attempts or refunds.
- bookings 1:0..1 reviews; each review refers to one hotel and should only be accepted for an eligible completed booking.

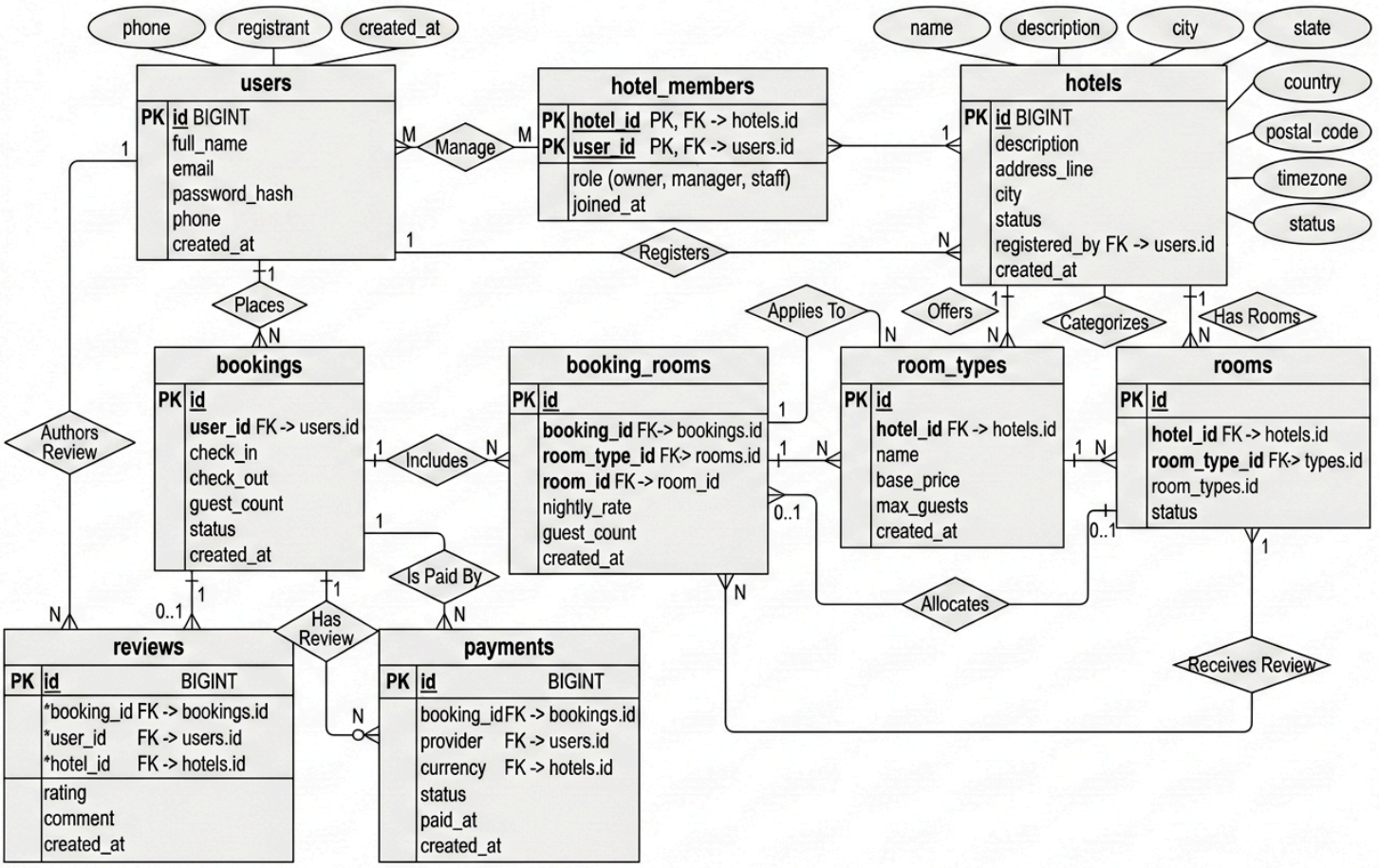
4. Integrity constraints and implementation notes

- Enforce tenant isolation in FastAPI: verify the authenticated user has access to the hotel for every hotel-scoped read/write.
- Add composite foreign keys or triggers to guarantee rooms.hotel_id matches room_types.hotel_id, booking_rooms.room_id matches booking_rooms.room_type_id, and reviews.hotel_id/user_id match the associated booking. Simple single-column FKs do not enforce these cross-column rules.
- Use a PostgreSQL daterange exclusion constraint for assigned rooms so active bookings cannot overlap. Exclude cancelled bookings, or model room occupancy in a dedicated allocation table.
- Validate guest counts against room_types.max_guests, total guest counts, booking status transitions, payment totals, and review eligibility in transactions or database constraints/triggers as appropriate.
- Add indexes on hotels(city, status), room_types(hotel_id), rooms(hotel_id, room_type_id, status), bookings(user_id, check_in), booking_rooms(booking_id), payments(booking_id), and reviews(hotel_id).
- Use CHECK constraints or PostgreSQL ENUMs for status/role values. CHECK constraints are easier to evolve during an early project.

5. Normalization and design justification

The schema is designed to approximately Third Normal Form (3NF): users, hotels, room types, physical rooms, bookings, payments, and reviews are separated into entities with their own primary keys. hotel_members resolves the many-to-many hotel/user membership and stores the role that belongs to that relationship. booking_rooms resolves a booking's multiple requested rooms. Hotel address fields are stored with the hotel because each hotel has one primary address in this initial scope. nightly_rate is intentionally copied into booking_rooms as a historical snapshot, not derived from the current room-type price. If the project later needs seasonal pricing, taxes, discounts, cancellation policies, or multiple hotel addresses, add separate normalized tables for those concepts.

6. Enhanced Entity-Relationship (EER) diagram



- **Entities (Rectangles):** Represent key domain concepts including **users**, **hotels**, **hotel_members**, **room_types**, **rooms**, **bookings**, **booking_rooms**, **payments**, and **reviews**.
- **Attributes (Ovals):** Connected to their respective entity rectangles. Primary key attributes (e.g., *id* on **users**, *hotel_id* & *user_id* on **hotel_members**) are underlined.
- **User–Hotel Relationships (Diamonds):** A diamond for *registers* connects **users** (1) to **hotels** (N). A diamond for *belongs_to* links **users** and **hotels** via the **hotel_members** associative entity in an M:N structure.
- **Hotel–Room Relationships (Diamonds):** Diamonds for *offers* and *contains* connect **hotels** (1) to **room_types** (N) and **rooms** (N). A diamond for *classified_as* links **rooms** (N) to **room_types** (1).
- **Booking Relationships (Diamonds):** A diamond for *places* connects **users** (1) to **bookings** (N). Diamonds for *includes*, *generates*, and *receives* connect **bookings** to **booking_rooms** (1:N), **payments** (1:N), and **reviews** (1:0..1).
- **Room Assignment Relationships (Diamonds):** Diamonds for *requests* and *assigns* connect **booking_rooms** to **room_types** (N:1) and optionally physical **rooms** (N:0..1).

7. Stack and architecture (high level)

The frontend is developed using Next.js, with FastAPI handling backend REST APIs and PostgreSQL managing data storage. The frontend supports hotel browsing, search, room selection, booking checkout, and customer and manager dashboards. The backend is responsible for authentication, authorization, input validation, room availability, booking management, payment integration, and tenant-level access control. PostgreSQL serves as the primary source of truth. Sensitive credentials are managed through environment variables, while Alembic is used to handle database schema changes through migrations, ensuring consistency across deployments.

8. Proposed Application Workflows

- **Hotel Search and Room Selection:** Customers search hotels by location, view available room types, compare prices, and select rooms based on guest capacity and check-in/check-out dates.
- **Booking and Payment:** Customers create bookings, select one or more rooms, confirm availability, and initiate payment. Booking and payment records are maintained in the database.
- **Booking Management and Reviews:** Customers view their bookings and cancel eligible reservations. After completing an eligible stay, customers can submit a hotel review and rating.